

UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingeniería de Sistemas
Informáticos



UNIVERSIDAD
POLITÉCNICA
DE MADRID



Máster
Deep Learning
Universidad Politécnica de Madrid

Trabajo Fin de Máster

Sistema LLM para análisis financiero histórico
mediante generación automática de código

MÁSTER DE FORMACIÓN PERMANENTE
EN DEEP LEARNING

Presentado por:

Carlos Ruiz Oyarzun

Bajo la dirección de:

Adrián Girón Jiménez

Alejandro Delgado Peribáñez

Madrid, 2026

Resumen

Este Trabajo Fin de Máster propone el desarrollo incremental de un sistema orientado al análisis financiero histórico con modelos de lenguaje. El sistema parte de una consulta de usuario, datos de mercado almacenados en CSV y metadatos como tickers, fechas o intervalo temporal. A partir de esa entrada, el prototipo valida la solicitud, utiliza un agente planificador para definir el análisis, un agente generador para producir código Python ejecutable, valida dicho código, lo lanza en un proceso controlado y transforma la salida estructurada en una respuesta comprensible mediante un agente interpretador.

El MVP actual soporta varias familias de consultas, desde análisis simples de crecimiento hasta comparativas con tablas, datos estructurados y explicaciones para usuario final. La evaluación combina pruebas funcionales, escenarios de error controlado y una batería A/B/C equilibrada con cinco consultas por nivel. Los resultados muestran que el sistema puede mantener el cálculo financiero bajo control programático, conservar trazabilidad sobre los artefactos y limitar la respuesta al análisis histórico, sin predicción futura ni recomendación de inversión.

Palabras clave: análisis financiero, modelos de lenguaje, generación de código, ejecución controlada, clientes LLM compatibles.

Abstract

This Master's Thesis proposes the incremental development of a language-model-driven system for historical financial analysis. The system starts from a user query, market data stored in CSV files and metadata such as tickers, dates or time interval. From this input, the prototype validates the request, uses a first LLM agent to plan the analysis, a second agent to generate executable Python code, validates that code, runs it in a controlled process and transforms the structured output into an interpretable response through a third interpretation agent.

The current MVP supports several families of queries, ranging from simple growth analysis to comparisons with tables, structured data and final-user explanations. The evaluation combines functional tests, controlled error scenarios and a balanced A/B/C battery with five queries per level. The results show that the system can keep financial computation under programmatic control, preserve traceability over generated artifacts and limit the response to historical analysis, without future prediction or investment recommendation.

Keywords: financial analysis, language models, code generation, controlled execution, compatible LLM clients.

Índice general

Resumen	I
Abstract	II
1. Introducción	1
1.1. Contexto y definición del problema	1
1.2. Motivación	2
1.3. Objetivos	3
1.4. Alcance	4
1.5. Estructura de la memoria	4
2. Estado de la cuestión	5
2.1. Enfoque de la revisión	5
2.2. Modelos de lenguaje en finanzas	5
2.3. Agentes LLM aplicados a finanzas	6
2.4. LLM con herramientas y ejecución	6
2.5. Generación de código para análisis de datos	7
2.6. Modelos multimodales y alcance del prototipo	7
2.7. Fiabilidad, alucinaciones y trazabilidad	8
2.8. Comparativa y posicionamiento de la propuesta	9
3. Datos y material de trabajo	10
3.1. Fuente de datos y datos obtenidos	10
3.2. Catálogo de consultas y proceso de descarga	11
3.3. Estructura de los CSV	13
3.4. Análisis exploratorio de datos	14
3.5. Calidad, cobertura y limitaciones	15
3.6. Material generado por el flujo y trazabilidad	15
4. Metodología	18
4.1. Enfoque de desarrollo	18
4.2. Proceso incremental	18

4.3. Criterios de validación	19
4.4. Niveles de dificultad	19
4.5. Métricas de evaluación	20
4.6. Revisión humana	20
5. Diseño del sistema	21
5.1. Arquitectura general	21
5.2. Nodos del flujo	22
5.3. Contrato del código generado	23
5.4. Gestión de errores	23
6. Implementación	24
6.1. Estructura del código	24
6.2. Repositorio y organización del trabajo	24
6.3. Contratos de datos	25
6.4. Pipeline LLM	26
6.5. Control de seguridad	27
6.6. Reparación y compactación	27
6.7. Pruebas	28
7. Integración de modelos de lenguaje	29
7.1. Papel del LLM en el sistema	29
7.2. Contrato de prompts	29
7.3. Niveles A/B/C	30
7.4. Cliente compatible con Chat Completions	31
7.5. Gestión de errores	31
7.6. Credenciales y despliegue	31
8. Resultados y evaluación	32
8.1. Objetivo	32
8.2. Batería A/B/C	32
8.3. Métricas técnicas	33
8.4. Resultados observados	33
8.5. Limitaciones	34
8.6. Síntesis	34
9. Aspectos éticos, seguridad y uso responsable	35
9.1. Delimitación financiera del sistema	35
9.2. Riesgos de interpretación	35
9.3. Seguridad en la generación y ejecución de código	35
9.4. Privacidad y datos	36

9.5. Sostenibilidad, costes y recursos	36
9.6. Limitaciones de uso	36
10. Conclusiones y trabajo futuro	37
Referencias	39
A. Anexos	42
A.1. Ejecución del MVP	42
A.2. Configuración de API	42
A.3. Batería de consultas A/B/C	43
A.4. Prompts usados por los agentes	44
A.4.1. Agente 1: planificación	44
A.4.2. Agente 2: generación de código	44
A.4.3. Agente 3: interpretación	44
A.5. Trazabilidad de una ejecución	44
A.6. Métricas subjetivas futuras	44

Índice de tablas

2.1. Comparativa entre trabajos relacionados y la propuesta del TFM.	9
3.1. Datos obtenidos y finalidad de cada conjunto dentro del MVP.	11
3.2. Estructura semántica de una consulta financiera antes de su procesamiento.	13
3.3. Campos OHLCV presentes en los CSV de mercado.	13
3.4. Resumen de cobertura y calidad del conjunto de datos.	14
3.5. Muestra de cobertura temporal obtenida en el EDA.	14
3.6. Métricas exploratorias por serie utilizadas para contextualizar los casos de prueba.	15
3.7. Material generado por el flujo y papel de cada agente.	17
4.1. Criterios de validación aplicados al flujo del prototipo.	19
5.1. Responsabilidades principales de la arquitectura.	22
6.1. Módulos principales de la implementación.	24
6.2. Contrato operativo del código Python generado por el LLM.	26
6.3. Comprobaciones principales del validador de seguridad.	27
7.1. Responsabilidades de los tres agentes LLM.	30
8.1. Estructura de la batería A/B/C de 15 consultas.	32
8.2. Lectura de los estados principales de evaluación.	34
A.1. Batería de 15 consultas por nivel de profundidad.	43

Índice de figuras

1.1. Mapa conceptual de relaciones entre inteligencia artificial, aprendizaje automático, redes neuronales, aprendizaje profundo e inteligencia artificial generativa. Fuente: Analytics Vidhya [2].	2
5.1. Arquitectura general del sistema desde la consulta del usuario hasta la respuesta trazable.	21

Capítulo 1

Introducción

1.1. Contexto y definición del problema

La inteligencia artificial ha evolucionado desde sistemas basados en reglas y técnicas estadísticas hasta modelos capaces de aprender representaciones complejas a partir de grandes volúmenes de datos. Dentro de esta evolución, el aprendizaje automático permitió entrenar modelos a partir de ejemplos; el aprendizaje profundo amplió esa capacidad mediante redes neuronales con muchas capas; y la inteligencia artificial generativa incorporó modelos capaces de producir texto, código, imágenes u otros artefactos a partir de instrucciones. Un punto de inflexión especialmente relevante fue la arquitectura Transformer, introducida en el trabajo *Attention Is All You Need* [1]. Su mecanismo de atención facilitó el entrenamiento de modelos de lenguaje de gran escala y se convirtió en una de las bases técnicas sobre las que se apoyan los LLM actuales.

Este TFM se sitúa dentro de esa evolución, pero no desde la perspectiva de entrenar un nuevo modelo. El foco está en integrar modelos de lenguaje ya disponibles dentro de un sistema software verificable. En concreto, el proyecto explora cómo un LLM puede ayudar a interpretar consultas financieras, generar código Python e interpretar resultados, manteniendo el cálculo bajo control programático y conservando artefactos auditables.

El análisis financiero histórico combina carga de datos, validación de entradas, selección del tipo de análisis, cálculo de métricas, generación de código, interpretación de resultados y comunicación final al usuario. En muchos prototipos basados en modelos de lenguaje estas fases se mezclan dentro de una única llamada generativa, lo que dificulta la trazabilidad del resultado y aumenta el riesgo de errores silenciosos.

Este trabajo plantea una aproximación controlada: el sistema parte de una entrada estructurada con consulta, tickers, fechas y rutas CSV; valida los datos mínimos; usa un agente planificador para definir el análisis; usa un agente generador para producir código Python; valida el contrato de seguridad del script; ejecuta el código en un

entorno controlado; y finalmente usa un agente interpretador para explicar las métricas obtenidas.

Para situar el alcance tecnológico del trabajo, la Figura 1.1 resume la relación entre inteligencia artificial, aprendizaje automático, aprendizaje profundo e inteligencia artificial generativa. El TFM no se centra en entrenar modelos ni en desarrollar nuevas arquitecturas neuronales, sino en integrar modelos de lenguaje dentro de un sistema trazable de análisis financiero histórico.

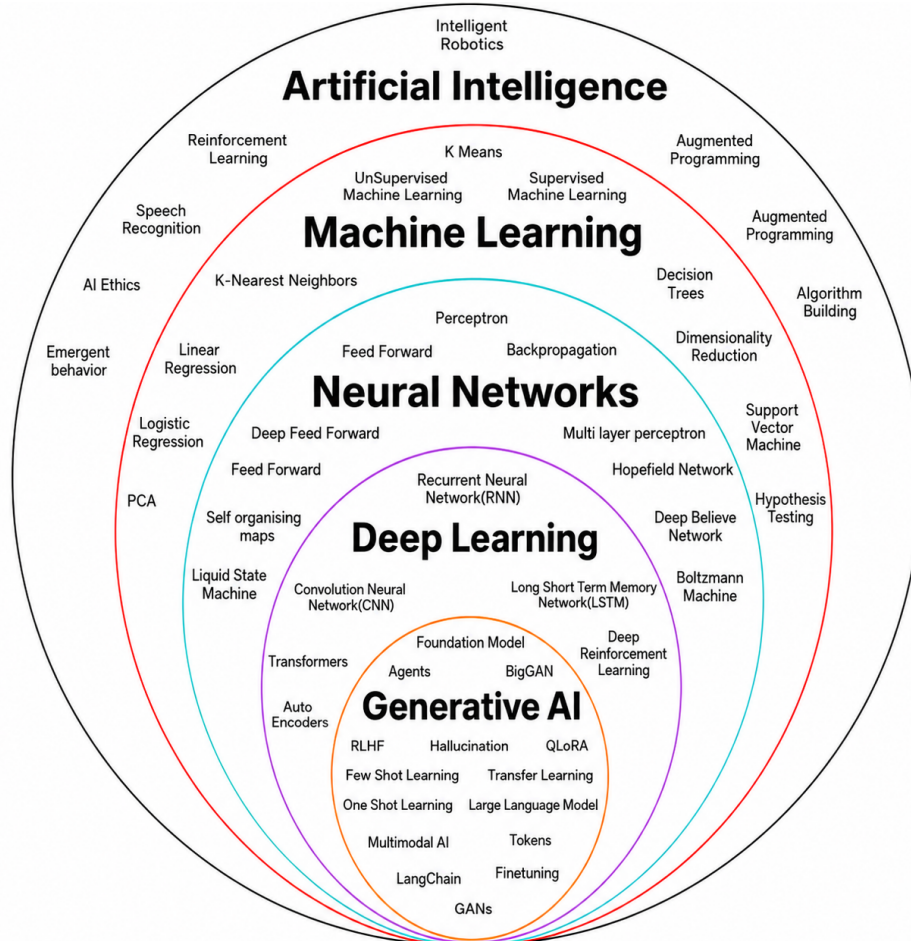


Figura 1.1: Mapa conceptual de relaciones entre inteligencia artificial, aprendizaje automático, redes neuronales, aprendizaje profundo e inteligencia artificial generativa. Fuente: Analytics Vidhya [2].

1.2. Motivación

La motivación principal del proyecto es explorar cómo un flujo LLM con ejecución controlada puede aportar orden, modularidad y robustez a un sistema de análisis financiero asistido por inteligencia artificial. La aplicación de modelos de lenguaje al dominio financiero resulta atractiva por su capacidad para interpretar peticiones, generar código y adaptar el lenguaje al usuario, pero también exige mecanismos de control,

especialmente cuando la respuesta final puede confundirse con una recomendación financiera.

Además, el proyecto nace de un interés personal por la intersección entre inteligencia artificial, programación y análisis financiero. La disponibilidad de modelos generativos abre la posibilidad de construir prototipos útiles con una rapidez que hace pocos años era difícil de imaginar. Sin embargo, precisamente por esa facilidad, resulta importante defender un enfoque responsable: las salidas del modelo deben revisarse, contrastarse con datos y situarse dentro de los límites del conocimiento experto. En este TFM, esa idea se traduce en una arquitectura donde el LLM ayuda, pero no sustituye la validación, la ejecución controlada ni la revisión crítica de resultados.

El interés del trabajo no está en construir un producto financiero completo, sino en estudiar si un prototipo incremental puede convertir consultas históricas en cálculos trazables y explicaciones prudentes. Por ello se evita que el LLM actúe como una caja negra que responde directamente al usuario: el sistema separa planificación, generación de código e interpretación final, valida el código antes de ejecutarlo y captura los resultados en formato estructurado. Si el modelo no está configurado o falla, el sistema informa el error y no genera una respuesta alternativa sin modelo.

En esta memoria se utiliza el término MVP como *prototipo mínimo viable* dentro de una evolución por etapas, no como producto comercial cerrado. El primer MVP validó consultas básicas de crecimiento y comparación; el segundo amplió la cobertura analítica; y la versión actual incorpora tres agentes diferenciados, generación de código controlada, validación de seguridad, ejecución aislada y artefactos auditables. Esta evolución se detalla en la Sección 4.2.

1.3. Objetivos

El objetivo general del trabajo es desarrollar y validar un prototipo basado en LLM capaz de ejecutar análisis financieros históricos a partir de entradas estructuradas y datos en CSV.

Los objetivos específicos son:

- Definir un flujo que separe validación, planificación, generación de código, control de seguridad, ejecución e interpretación final.
- Diseñar un agente planificador capaz de interpretar la consulta, identificar el nivel A/B/C y construir un plan estructurado.
- Diseñar un agente generador capaz de producir código Python a partir del plan analítico y del mensaje original del usuario.

- Validar el código generado mediante contrato de script, importaciones permitidas y salida JSON.
- Ejecutar el código en un entorno controlado y conservar artefactos auditables.
- Diseñar un agente interpretador que redacte la respuesta final a partir de métricas ya calculadas, sin añadir datos externos ni recomendaciones de inversión.
- Cubrir consultas de distinta profundidad, desde respuestas básicas hasta salidas con tablas, datos visuales y comparativas detalladas.
- Evaluar el MVP mediante casos funcionales, escenarios de error controlado y una batería A/B/C equilibrada.

1.4. Alcance

El sistema se limita al análisis histórico-descriptivo de activos financieros a partir de datos ya descargados. No realiza predicción futura, recomendación de inversión ni análisis fundamental complejo. Esta restricción es deliberada: el foco del TFM no es construir un asesor financiero, sino estudiar una arquitectura LLM trazable para análisis histórico con generación y ejecución controlada de código.

1.5. Estructura de la memoria

La memoria se organiza siguiendo la lógica de desarrollo del proyecto. El Capítulo 2 presenta el estado de la cuestión, con especial atención a modelos de lenguaje en finanzas, uso de herramientas externas, generación de código, multimodalidad y trazabilidad. El Capítulo 3 describe los datos utilizados, su procedencia, su organización en CSV y los artefactos generados por el flujo. El Capítulo 4 expone la metodología incremental seguida durante el desarrollo del MVP.

Los capítulos centrales describen la arquitectura y la implementación. El Capítulo 5 presenta el diseño general del sistema. El Capítulo 6 detalla los módulos de código, el contrato de ejecución, el control de seguridad y la compactación de salidas. El Capítulo 7 explica la integración de modelos de lenguaje y la separación entre agentes. El Capítulo 8 recoge el planteamiento de evaluación y la batería A/B/C. Finalmente, el Capítulo 9 aborda consideraciones éticas y de seguridad, el Capítulo 10 resume conclusiones y trabajo futuro, y los anexos incluyen configuraciones, batería de consultas, prompts y ejecuciones resumidas.

Capítulo 2

Estado de la cuestión

2.1. Enfoque de la revisión

El objetivo de este capítulo es situar el TFM dentro del contexto actual de los modelos de lenguaje aplicados al análisis financiero y a la automatización de tareas de datos. La revisión no pretende cubrir todo el campo de la inteligencia artificial financiera, sino seleccionar aquellas líneas de trabajo que ayudan a justificar las decisiones del proyecto: uso de LLM en finanzas, agentes con herramientas externas, generación de código, salidas visuales, fiabilidad y trazabilidad.

Esta delimitación es importante porque el sistema desarrollado no consiste en entrenar un nuevo modelo financiero ni en construir un asesor de inversión. La propuesta se centra en una arquitectura de integración: una consulta de usuario se valida, se interpreta mediante LLM, se transforma en código Python, se revisa antes de ejecutarse y se convierte en una respuesta final basada en resultados ya calculados. Por tanto, el estado de la cuestión se orienta a comparar el TFM con trabajos que combinan lenguaje natural, dominio financiero, herramientas, código y control de ejecución.

2.2. Modelos de lenguaje en finanzas

El uso de procesamiento del lenguaje natural en finanzas es anterior a los modelos generativos actuales. FinBERT adapta modelos tipo BERT al dominio financiero y muestra que el lenguaje de mercado tiene particularidades suficientes como para justificar modelos especializados, especialmente en tareas de sentimiento financiero [3]. Con la aparición de LLM de mayor escala, BloombergGPT y FinGPT ampliaron el foco hacia modelos entrenados o adaptados con datos financieros, documentación corporativa, noticias, series de mercado y otros recursos específicos del sector [4], [5].

Estos trabajos son relevantes porque demuestran que el dominio financiero se beneficia de modelos capaces de comprender terminología, contexto y relaciones propias del

sector. Sin embargo, se sitúan en una escala distinta a la de este TFM. BloombergGPT y FinGPT se centran en crear o adaptar modelos; este proyecto, en cambio, integra modelos disponibles mediante un cliente común y estudia cómo encajarlos en un flujo verificable de análisis histórico.

La diferencia también afecta al alcance funcional. El sistema de esta memoria no analiza noticias, documentos corporativos ni sentimiento de mercado, y tampoco intenta predecir precios. Trabaja con datos históricos locales en CSV y calcula métricas descriptivas. Por ello, la aportación no se encuentra en el entrenamiento del modelo, sino en el diseño de un proceso que separa interpretación, cálculo, validación y explicación.

2.3. Agentes LLM aplicados a finanzas

En trabajos recientes aparecen sistemas financieros con componentes más próximos a agentes. FinRobot propone un agente de investigación y valoración de renta variable apoyado en LLM [6]. TradingAgents plantea un marco multiagente orientado a tareas de trading, donde distintos agentes simulan roles de análisis, debate o decisión [7]. Estos enfoques reflejan una tendencia clara: los LLM dejan de actuar únicamente como generadores de texto y pasan a formar parte de sistemas con roles, herramientas, memoria de trabajo y salidas especializadas.

La relación con este TFM es directa, pero también conviene marcar la diferencia. El prototipo desarrollado no busca tomar decisiones de trading ni realizar valoración fundamental completa. Tampoco implementa una sociedad de agentes autónomos que negocien entre sí. La memoria utiliza el término agente en sentido funcional: un primer bloque LLM interpreta la consulta, planifica y genera código; un segundo bloque LLM interpreta resultados calculados para el usuario final. Esta separación permite estudiar la utilidad de los agentes sin desplazar el control del sistema hacia decisiones financieras autónomas.

Desde el punto de vista académico, esta decisión reduce ambición comercial pero aumenta trazabilidad. El interés no es que el modelo actúe libremente sobre el mercado, sino comprobar si un flujo acotado puede convertir lenguaje natural en cálculos reproducibles y explicaciones prudentes.

2.4. LLM con herramientas y ejecución

El diseño técnico del sistema se relaciona con la literatura sobre modelos capaces de razonar y actuar con herramientas externas. ReAct introdujo la combinación de razonamiento y actuación, de modo que el modelo alterna pasos de pensamiento con acciones observables [8]. Toolformer mostró que los modelos pueden aprender a decidir cuándo

llamar a herramientas externas [9]. Las revisiones sobre agentes autónomos basados en LLM describen esta evolución como un paso desde modelos conversacionales hacia sistemas capaces de planificar, actuar y observar resultados en entornos externos [10].

El TFM adopta esta idea general, pero evita que el modelo decida libremente qué herramienta usar o qué acción ejecutar. El flujo está fijado por el propio sistema: validación de entrada, análisis LLM, generación de código, control de seguridad, ejecución e interpretación final. Esta arquitectura limita la autonomía del modelo, pero facilita la depuración y permite revisar cada artefacto intermedio.

En el contexto financiero, esta restricción es especialmente relevante. Una respuesta generada directamente por un LLM puede sonar convincente aunque no esté respaldada por cálculos reales. Al obligar al sistema a pasar por código ejecutado y salida JSON, el prototipo aproxima el comportamiento del modelo a una herramienta de asistencia controlada, no a una fuente opaca de conclusiones.

2.5. Generación de código para análisis de datos

La generación automática de código es una de las capacidades más útiles de los LLM modernos. El trabajo sobre Codex y HumanEval mostró que los modelos entrenados con código pueden sintetizar programas a partir de descripciones en lenguaje natural [11]. En tareas de ciencia de datos, DS-1000 propuso un banco de prueba específico para evaluar generación de código en escenarios realistas con librerías habituales de análisis [12].

Estos trabajos justifican que un sistema de análisis financiero pueda usar un LLM para producir código Python adaptado a la consulta. No obstante, también evidencian un riesgo central: un programa plausible no es necesariamente correcto, seguro ni ejecutable. En este TFM, la generación de código no se trata como una salida final, sino como un artefacto que debe superar controles antes de ejecutarse.

El sistema exige un contrato mínimo: el script debe leer la entrada normalizada, usar librerías permitidas, producir una salida JSON y no realizar acciones fuera del alcance autorizado. Esta capa convierte la generación de código en una fase verificable del workflow. El resultado final no depende solo de la fluidez del modelo, sino de que el código generado pueda ejecutarse y producir métricas coherentes con los datos.

2.6. Modelos multimodales y alcance del prototipo

Los modelos multimodales amplían los modelos de lenguaje textual para trabajar con más de un tipo de entrada o salida. En lugar de recibir únicamente texto, pueden combinar instrucciones con imágenes, vídeo, audio o documentos, según la arquitectura

y el proveedor. Flamingo es un ejemplo representativo de esta línea, al conectar modelos visuales y lingüísticos para procesar secuencias intercaladas de imágenes, vídeos y texto [13]. En sistemas más recientes, GPT-4V describe capacidades para interpretar imágenes junto con instrucciones textuales [14].

Esta línea resulta relevante como contexto general, pero no forma parte de la evaluación principal del prototipo. El sistema se centra en texto, planes JSON, código Python y resultados estructurados calculados sobre CSV locales. La revisión subjetiva de claridad, formato o utilidad explicativa se conserva como una línea futura de mejora.

Sin embargo, el prototipo desarrollado en este TFM no utiliza modelos multimodales dentro del flujo principal. El LLM trabaja con texto, planes JSON, código Python y salidas estructuradas. Esta aclaración evita sobredimensionar el alcance del sistema y deja la multimodalidad como una línea futura razonable.

2.7. Fiabilidad, alucinaciones y trazabilidad

Una limitación conocida de los modelos generativos es la producción de respuestas plausibles pero no fieles a los datos, fenómeno habitualmente denominado alucinación [15]. En tareas de conocimiento, la generación aumentada con recuperación propuso reducir este problema apoyando la generación en fuentes externas recuperadas explícitamente [16]. Aunque este TFM no implementa un sistema RAG documental, sí adopta una lógica de anclaje: la respuesta final debe apoyarse en datos locales, código ejecutado y resultados estructurados.

En análisis financiero, la trazabilidad no es un detalle técnico secundario. Si una respuesta informa de rentabilidad, volatilidad o drawdown, debe poder revisarse qué datos se usaron, qué cálculo se hizo y qué limitaciones tiene la interpretación. Además, el lenguaje final debe evitar que una descripción histórica se lea como predicción futura o recomendación de inversión.

Por este motivo, la memoria separa cada responsabilidad: los datos se documentan en el capítulo correspondiente, el plan se genera antes del código, el script se valida antes de ejecutarse, la salida se conserva como artefacto y la interpretación final se limita a explicar métricas calculadas. Esta separación es la base metodológica que diferencia el prototipo de una interacción directa con un chatbot generalista.

2.8. Comparativa y posicionamiento de la propuesta

La Tabla 2.1 resume la relación entre las líneas revisadas y el sistema desarrollado en este TFM. La comparación permite identificar qué se toma como inspiración y qué queda fuera del alcance. La propuesta no compite con modelos financieros entrenados a gran escala ni con marcos de trading multiagente; su aportación se sitúa en la integración controlada de LLM, generación de código y análisis histórico reproducible.

Trabajo o línea	Dominio	Enfoque	Relación con este TFM
FinBERT	PLN financiero	Modelo especializado para sentimiento financiero [3].	Muestra la utilidad de adaptar lenguaje al dominio financiero, pero no aborda generación controlada de código ni ejecución de métricas históricas.
BloombergGPT y FinGPT	LLM financieros	Modelos grandes entrenados o adaptados con datos financieros [4], [5].	Se centran en el modelo; este TFM integra modelos ya disponibles dentro de un flujo software verificable.
FinRobot y TradingAgents	Agentes financieros	Agentes LLM para investigación, valoración o marcos multiagente de trading [6], [7].	Inspiran la separación de roles, pero el prototipo no toma decisiones de inversión ni implementa un agente autónomo de trading.
ReAct y Toolformer	LLM con herramientas	Razonamiento combinado con acciones y llamadas a herramientas [8], [9].	Justifican el uso de herramientas externas; el TFM fija un flujo más controlado para mejorar trazabilidad y seguridad.
Codex y DS-1000	Generación de código	Síntesis y evaluación de código funcional o de ciencia de datos [11], [12].	Respaldan la generación de código del MVP y la necesidad de validación estricta antes de ejecutar.
Flamingo y GPT-4V	Multimodalidad	Modelos capaces de combinar texto con imágenes, vídeo, audio o documentos [13], [14].	Sitúan la revisión multimodal como posible evolución futura; el flujo actual no envía imágenes al modelo.
Alucinaciones y RAG	Fiabilidad	Reducción de respuestas no fieles mediante anclaje en fuentes o datos externos [15], [16].	El TFM adopta una lógica de anclaje en CSV locales, código ejecutado y salida JSON, aunque no implemente RAG documental.

Tabla 2.1: Comparativa entre trabajos relacionados y la propuesta del TFM.

En consecuencia, el posicionamiento del trabajo es deliberadamente intermedio. No se plantea un nuevo LLM financiero, ni un sistema autónomo de inversión, ni una plataforma multimodal completa. Se propone un prototipo trazable para análisis financiero histórico donde el modelo aporta interpretación, generación de código y redacción final, mientras el sistema conserva el control sobre validación, ejecución y artefactos. Esta combinación es la que da sentido al diseño descrito en los capítulos posteriores.

Capítulo 3

Datos y material de trabajo

3.1. Fuente de datos y datos obtenidos

El sistema trabaja con datos financieros históricos descargados previamente mediante `yfinance` [17] y almacenados en ficheros CSV. Yahoo Finance publica información de mercado, fichas de activos y series históricas para acciones, fondos cotizados, índices, divisas, criptomonedas y materias primas. La librería `yfinance` permite acceder de forma programática a parte de esta información disponible en Yahoo Finance [18], incluyendo campos habituales como apertura, máximo, mínimo, cierre, cierre ajustado y volumen. En este trabajo se utiliza como herramienta práctica de descarga para un contexto académico, no como proveedor garantizado para producción.

Cada CSV se conserva junto a un fichero de metadatos en formato JSON. Este metadato registra la consulta original, el objetivo analítico, los símbolos financieros implicados, el intervalo temporal y los parámetros de descarga. La decisión permite separar la fase de obtención de datos de la fase de análisis: el MVP no depende de que la API externa esté disponible durante cada ejecución, sino que opera sobre un conjunto de datos congelado y revisable.

La Tabla 3.1 muestra los datos obtenidos y utilizados como base del TFM. No se incluyen todos los nombres de fichero en la tabla para evitar una presentación demasiado técnica; los nombres completos se mantienen en `data/raw` y en el resumen procesado `data/processed/eda_resumen_ficheros.csv`.

Caso de datos	Activos	Cobertura	Uso en el MVP
NVDA cinco años	NVDA	2021–2026, diario	Crecimiento de precio y validación de respuestas de Nivel A.
NVDA frente a AMD	NVDA, AMD	2024–2026, diario	Comparación entre activos y cálculo de rentabilidad relativa.
AAPL tres meses	AAPL	2025–2026, diario	Visión descriptiva de un activo y análisis técnico básico.
QQQ frente a SPY	QQQ, SPY	2024, diario	Retorno, riesgo, drawdown y comparación estructurada.
S&P 500 desde 2020	^GSPC	2020–2026, diario	Serie amplia de mercado para pruebas de histórico largo.
Bitcoin 2024	BTC-USD	2024, diario	Activo cripto con alta variabilidad.
EUR/USD horario	EURUSD=X	10 días, 1h	Ejemplo de divisa con frecuencia intradía.
Oro horario	GC=F	1 semana, 1h	Materia prima con frecuencia intradía.

Tabla 3.1: Datos obtenidos y finalidad de cada conjunto dentro del MVP.

La variedad de activos permite comprobar que el sistema no está limitado a una acción concreta. También fuerza al flujo LLM a trabajar con horizontes distintos, desde consultas diarias de varios años hasta series horarias de pocos días.

3.2. Catálogo de consultas y proceso de descarga

El punto de partida del conjunto de datos es un catálogo de consultas de referencia. Cada entrada define la consulta, los tickers, el rango temporal o período relativo, el intervalo de observación y los posibles avisos. Este catálogo actúa como contrato entre la fase de preparación de datos y el flujo de análisis.

El proceso utilizado para construir el conjunto de datos puede resumirse en cuatro pasos:

1. Definir o revisar la consulta en el catálogo, por ejemplo una comparativa entre QQQ y SPY o una descarga de NVDA a cinco años.
2. Transformar esa consulta en parámetros de descarga: lista de tickers, **start**, **end**, **period** e **interval**.

3. Descargar las series con `yfinance` y guardar el resultado en `data/raw` como CSV, junto a un `.metadata.json` con los parámetros usados.
4. Normalizar los CSV, generar tablas procesadas y producir resúmenes exploratorios auxiliares.

En el repositorio actual, los CSV crudos ya están congelados. Por tanto, para reproducir la parte documentada de esta memoria no es necesario volver a llamar a Yahoo Finance: basta con conservar los datos crudos y regenerar las tablas procesadas si fuera necesario. Si se quisiera ampliar el conjunto de datos, el procedimiento correcto sería añadir primero la consulta al catálogo y después generar un nuevo CSV con su metadato asociado.

Cada consulta se transforma en una representación estructurada antes de entrar en el flujo LLM. El siguiente bloque muestra un ejemplo simplificado:

```
{
  "query": "Compárame QQQ y SPY en 2024 de forma clara,
           con tabla comparativa y serie normalizada",
  "tickers": ["QQQ", "SPY"],
  "csv_paths": [
    "data/raw/qqq_y_spy_desde_20240101_hasta_20241231.csv"
  ],
  "start": "2024-01-01",
  "end": "2024-12-31",
  "period": null,
  "interval": "1d",
  "warnings": []
}
```

La Tabla 3.2 resume el significado de los campos principales.

Elemento	Descripción	Ejemplo
Consulta original	Texto introducido o definido como caso de prueba.	Comparar Nvidia y AMD.
Objetivo analítico	Tipo de análisis que debe ejecutarse.	Comparación de activos.
Símbolos financieros	Activos sobre los que se realiza el análisis.	NVDA, AMD.
Rango temporal	Período relativo o fechas de inicio y fin.	Dos años.
Intervalo	Frecuencia de los datos históricos.	Diario.
Avisos	Observaciones detectadas al preparar la entrada.	Sin avisos.

Tabla 3.2: Estructura semántica de una consulta financiera antes de su procesamiento.

3.3. Estructura de los CSV

Los ficheros generados por `yfinance` contienen columnas de tipo OHLCV: apertura, máximo, mínimo, cierre, cierre ajustado y volumen. Cuando hay más de un ticker, las columnas se guardan con cabeceras multinivel: el primer nivel identifica el símbolo financiero y el segundo nivel identifica el campo de mercado. Por este motivo, los CSV no se leen como tablas planas con una sola fila de cabecera, sino con `header=[0, 1]`.

La Tabla 3.3 resume los campos usados por el sistema. La columna de cierre es la variable principal, porque de ella dependen las métricas de crecimiento, rentabilidad, volatilidad, drawdown e indicadores técnicos básicos.

Campo	Uso principal	Observación
Open	Precio de apertura.	Se conserva como contexto, aunque no es obligatorio en todas las respuestas.
High y Low	Rango máximo y mínimo.	Útiles para revisar amplitud diaria o intradía.
Close	Cálculo financiero central.	Base de rentabilidad, volatilidad, drawdown y medias móviles.
Adj Close	Precio ajustado.	Se conserva si aparece en la descarga original.
Volume	Actividad negociada.	Puede ser cero o menos informativo en divisas y algunos futuros.

Tabla 3.3: Campos OHLCV presentes en los CSV de mercado.

La estructura multinivel también explica por qué el proyecto incluye una capa de

normalización. El proceso exploratorio transforma cada combinación caso–ticker en formato largo, con columnas explícitas para identificador de caso, fichero fuente, ticker, intervalo, fecha y campos OHLCV. Este formato facilita auditar métricas por serie y evita que el código generado por el LLM tenga que resolver por sí solo todas las variantes de cabecera.

3.4. Análisis exploratorio de datos

El análisis exploratorio lee los CSV descargados con `yfinance`, interpreta las cabeceras multinivel, normaliza los datos a formato largo y genera resúmenes procesados por fichero y por serie.

El análisis confirma que el conjunto contiene 5.080 filas normalizadas, con un rango observado entre el 2 de enero de 2020 y el 17 de marzo de 2026. Los intervalos considerados son diario (1d) e intradía horario (1h).

Indicador	Resultado
Casos definidos en catálogo	10
CSV analizados	8
Series por símbolo y caso	10
Filas normalizadas	5.080
Nulos en precios de cierre	0
Duplicados por fecha	0

Tabla 3.4: Resumen de cobertura y calidad del conjunto de datos.

Caso	Tickers	Filas	Rango observado
AAPL tres meses	AAPL	60	2025-12-17 a 2026-03-16
NVDA frente a AMD	AMD, NVDA	500	2024-03-18 a 2026-03-16
NVDA cinco años	NVDA	1.255	2021-03-17 a 2026-03-16
Bitcoin 2024	BTC-USD	365	2024-01-01 a 2024-12-30
S&P 500 desde 2020	^GSPC	1.558	2020-01-02 a 2026-03-16
QQQ frente a SPY	QQQ, SPY	251	2024-01-02 a 2024-12-30

Tabla 3.5: Muestra de cobertura temporal obtenida en el EDA.

Además del resumen agregado, se calcularon métricas descriptivas por serie. La Tabla 3.6 muestra una selección de resultados. Se observan activos con crecimientos muy elevados, como NVDA en cinco años, series con caídas acumuladas significativas, como AMD en la comparativa con NVDA, y activos de menor volatilidad relativa,

como SPY. Esta diversidad obliga al sistema a redactar respuestas para escenarios con magnitudes y niveles de riesgo distintos.

Serie	Retorno total	Volatilidad anual	Máx. caída
NVDA, 5 años	1273.33 %	51.31 %	-66.36 %
BTC-USD, 2024	109.76 %	44.13 %	-26.18 %
NVDA, 2 años	107.13 %	49.42 %	-36.89 %
S&P 500, desde 2020	105.64 %	20.77 %	-33.93 %
QQQ, 2024	28.07 %	17.99 %	-13.56 %
SPY, 2024	24.45 %	12.63 %	-8.41 %
AMD, 2 años	3.11 %	55.10 %	-58.98 %
AAPL, 3 meses	-7.00 %	24.15 %	-10.07 %

Tabla 3.6: Métricas exploratorias por serie utilizadas para contextualizar los casos de prueba.

3.5. Calidad, cobertura y limitaciones

La ausencia de nulos en el precio de cierre es relevante porque las métricas principales del MVP dependen directamente de esta columna. También se comprobó la ausencia de duplicados por fecha en los ficheros analizados. Estas condiciones permiten ejecutar las consultas actuales sin introducir imputaciones adicionales.

Como limitación, las descargas basadas en períodos relativos, por ejemplo tres meses o cinco años, dependen de la fecha en la que se generó el CSV. Para una versión final completamente cerrada sería recomendable fijar fechas explícitas de inicio y fin en todos los casos. También debe considerarse que los datos proceden de Yahoo Finance mediante `yfinance`. Al tratarse de una herramienta abierta no afiliada ni avalada por Yahoo, su uso se considera adecuado para un contexto académico y experimental, pero en un entorno productivo habría que estudiar licencias, disponibilidad, estabilidad del proveedor y derechos de uso de los datos descargados [17].

3.6. Material generado por el flujo y trazabilidad

Además de los CSV históricos, el proyecto genera material intermedio durante cada ejecución. Este material es importante porque permite revisar cómo una consulta en lenguaje natural se transforma en una respuesta final. En un TFM no basta con indicar que el sistema responde: también debe poder explicarse qué entendió el modelo, qué cálculo se preparó, qué datos se usaron y qué resultado recibió el agente que redacta la explicación.

La Tabla 3.7 resume los artefactos conceptuales del flujo. La nomenclatura exacta puede evolucionar con la implementación, pero la separación de responsabilidades es estable: el agente planificador define el análisis, el agente generador produce código, la capa de ejecución produce resultados verificables y el agente interpretador convierte esos resultados en lenguaje natural.

Esta lectura también ayuda a distinguir material de trabajo de resultados finales. Las tablas procesadas y las notas auxiliares se conservan como evidencias para reconstruir decisiones de diseño, parámetros de descarga, estructura de los CSV, criterios de evaluación y salidas intermedias. No se presentan como conclusiones por sí mismas, sino como soporte metodológico para que el flujo sea reproducible y revisable.

Fase	Recibe	Genera	Utilidad en la memoria
Entrada normalizada	Consulta, tickers, rango temporal y rutas CSV.	<code>FinancialQueryInput</code> o estructura equivalente.	Fija qué datos puede usar el sistema y evita ambigüedades en la ejecución.
Agente planificador	Entrada normalizada y reglas de niveles A/B/C.	Plan de análisis con objetivo, métricas, salidas solicitadas y restricciones.	Permite evaluar si el modelo entendió la consulta antes de calcular nada.
Agente generador	Consulta original, entrada normalizada y plan analítico.	Script Python autocontenido ajustado al contrato.	Permite adaptar el cálculo a cada consulta sin perder validación.
Capa de ejecución	Código validado y datos históricos preparados.	Métricas, tablas y datos estructurados derivados de los CSV.	Separa cálculo reproducible de redacción en lenguaje natural.
Agente interpretador	Consulta original, plan y resultados estructurados.	Respuesta final comprensible, prudente y sin recomendación de inversión.	Permite valorar claridad, fidelidad a los datos y reconocimiento de limitaciones.
Trazabilidad	Artefactos anteriores y estado de ejecución.	Registro revisable de entrada, plan, cálculo, errores y respuesta.	Facilita auditoría, depuración y comparación entre modelos.

Tabla 3.7: Material generado por el flujo y papel de cada agente.

Capítulo 4

Metodología

4.1. Enfoque de desarrollo

La metodología combina desarrollo incremental de software y validación técnica del flujo. El objetivo no es entrenar un modelo financiero ni establecer una comparativa de servicios LLM, sino construir un prototipo capaz de transformar consultas financieras históricas en cálculos reproducibles y explicaciones prudentes.

La unidad de análisis es el flujo completo: entrada validada, planificación, generación de código, validación de seguridad, ejecución controlada e interpretación final. Esta decisión permite observar no sólo si una respuesta parece correcta, sino en qué punto del sistema se produce un acierto o un fallo.

4.2. Proceso incremental

El desarrollo se organizó en iteraciones:

1. Definir una entrada estructurada con consulta, tickers, fechas, intervalo y rutas a CSV locales.
2. Validar campos mínimos antes de invocar el modelo.
3. Separar el uso del LLM en tres agentes: planificación, generación de código e interpretación.
4. Exigir contratos JSON entre agentes y entre el código generado y el sistema.
5. Validar el código antes de ejecutarlo, limitando importaciones y operaciones permitidas.
6. Ejecutar el script sobre datos históricos locales y capturar salida estándar, errores y artefactos.

7. Compactar salidas largas antes de la interpretación final cuando sea necesario.
8. Revisar los resultados mediante métricas técnicas y, como línea futura, métricas subjetivas más reforzadas.

Este enfoque separa tres planos: datos, ejecución e interpretación. El plano de datos asegura que la consulta se resuelve sobre ficheros conocidos. El plano de ejecución controla el código generado. El plano interpretativo convierte resultados estructurados en una respuesta legible.

4.3. Criterios de validación

La validación del prototipo se organiza como una cadena de controles. Cada control detecta un tipo de fallo antes de que afecte a la respuesta final.

Nivel de validación	Qué comprueba	Riesgo que reduce
Entrada	Consulta no vacía, tickers, fechas, intervalo y CSV disponibles.	Evita lanzar el flujo con peticiones incompletas o imposibles de resolver.
Plan analítico	Nivel A/B/C, objetivo, métricas, columnas y salidas esperadas.	Reduce desalineamientos entre lo pedido y lo calculado.
Código generado	Función <code>main()</code> , lectura de <code>argv[1]</code> , importaciones permitidas y salida JSON.	Reduce riesgos de seguridad y salidas no parseables.
Ejecución	Código ejecutado, errores, métricas calculadas y salida estructurada.	Distingue fallo de generación, fallo de ejecución y resultado válido.
Interpretación	Uso exclusivo de resultados calculados, claridad, limitaciones y prudencia financiera.	Reduce alucinaciones y recomendaciones no permitidas.
Trazabilidad	Conservación de entrada, plan, código, salida y errores.	Permite auditar y repetir una ejecución.

Tabla 4.1: Criterios de validación aplicados al flujo del prototipo.

4.4. Niveles de dificultad

La batería de evaluación se organiza en tres niveles:

- **Nivel A:** consultas directas, sin formato especial. Se espera una selección mínima de métricas esenciales.
- **Nivel B:** consultas que piden más explicación, comparación, tabla o una salida visual principal. Se espera una salida estructurada más rica.
- **Nivel C:** consultas profesionales, multicriterio o con formato impuesto. Se espera coordinación de varias métricas, tablas, datos visuales si se solicitan y limitaciones.

La batería se equilibra con cinco consultas por nivel. Dentro de cada nivel, los ejemplos ascienden en complejidad: primero casos directos, después comparaciones, luego riesgo o ventanas temporales más exigentes, y finalmente consultas con formato más rígido.

4.5. Métricas de evaluación

La evaluación actual se centra en métricas técnicas:

- estado final del workflow;
- llegada o no a planificación, generación de código, ejecución e interpretación;
- errores de configuración, formato, validación, ejecución o interpretación;
- presencia de salida estructurada;
- ausencia de recomendaciones de inversión;
- tiempo de ejecución por caso.

Las métricas subjetivas quedan planteadas como trabajo futuro. Se consideran relevantes para valorar claridad, utilidad percibida, adecuación al nivel A/B/C y calidad comunicativa, pero requieren una rúbrica más sólida antes de usarse como evidencia principal. Por ello, en esta memoria no se presentan como resultados concluyentes.

4.6. Revisión humana

La revisión humana no sustituye a la validación técnica. Su papel es detectar si una respuesta técnicamente completa resulta insuficiente, ambigua o poco útil. En futuras iteraciones se reforzará mediante una rúbrica explícita y repetible, separando comprensión de la consulta, selección de métricas, fidelidad a la salida ejecutada y claridad de la interpretación.

Capítulo 5

Diseño del sistema

5.1. Arquitectura general

La arquitectura se diseña para evitar que el modelo de lenguaje responda directamente a una pregunta financiera sin cálculo verificable. El flujo separa tres responsabilidades generativas y varias capas deterministas de control.

El usuario formula una consulta en lenguaje natural. El sistema valida que existan tickers, fechas o periodo, intervalo y rutas a CSV locales. Después intervienen tres agentes LLM: el primero planifica el análisis, el segundo genera código Python y el tercero interpreta resultados ya calculados.

Entre la generación de código y la interpretación final se sitúa una capa de validación y ejecución controlada. Esta capa revisa el contrato del script, limita operaciones permitidas, ejecuta sobre datos históricos locales y captura la salida JSON. Así, las cifras no proceden de texto libre del modelo, sino de código ejecutado sobre datos trazables.

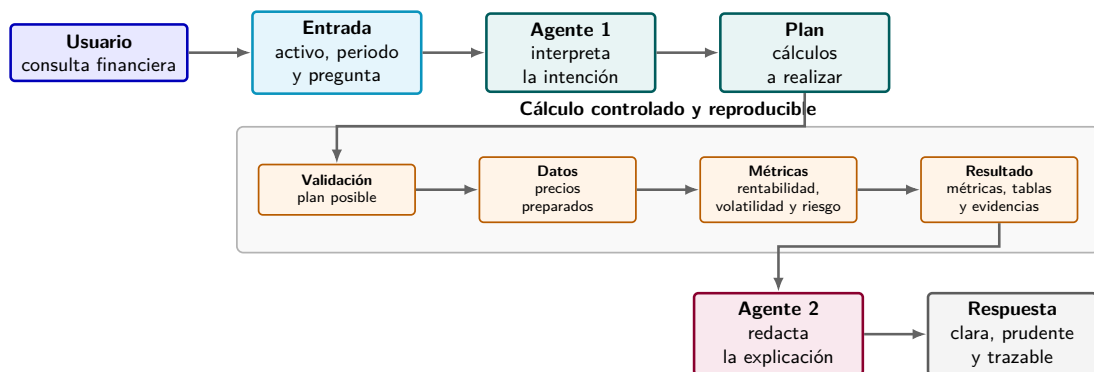


Figura 5.1: Arquitectura general del sistema desde la consulta del usuario hasta la respuesta trazable.

Bloque	Responsabilidad	Salida esperada
Entrada validada	Comprobar consulta, activos, fechas, intervalo y CSV.	Estructura de entrada procesable.
Agente 1	Planificar: interpretar la consulta, elegir nivel A/B/C y definir métricas y salidas.	Plan analítico JSON, sin cifras inventadas.
Agente 2	Generar código Python ajustado al contrato.	Script autocontenido que calcula sobre CSV locales.
Validación y ejecución	Revisar seguridad, ejecutar y capturar salida.	JSON con métricas, tablas, datos visuales si proceden y limitaciones.
Agente 3	Explicar resultados ya calculados.	Respuesta final en español, prudente y sin recomendación de inversión.

Tabla 5.1: Responsabilidades principales de la arquitectura.

La trazabilidad es una propiedad transversal. El sistema conserva la pregunta, el plan, el código generado, la salida ejecutada, los errores y la respuesta final. Esta información permite repetir pruebas y localizar en qué fase aparece un problema.

5.2. Nodos del flujo

El flujo implementado puede entenderse como una cadena de nodos:

- **Validación de entrada:** rechaza consultas vacías, fechas inválidas, ausencia de tickers o CSV inexistentes.
- **Agente 1: planificación:** identifica nivel A/B/C, objetivo analítico, métricas, columnas necesarias y salidas esperadas.
- **Agente 2: generación de código:** produce un script Python completo que lee un payload, calcula y emite JSON.
- **Validación de seguridad:** comprueba estructura, importaciones, lectura de entrada y salida JSON antes de ejecutar.
- **Reparación asistida:** si el script falla o se rechaza, puede pedirse una versión corregida que vuelve a validarse.
- **Ejecución controlada:** ejecuta el script en proceso separado sobre datos locales.

- **Compactación:** reduce salidas largas antes de enviarlas al agente interpretador, manteniendo la salida completa como evidencia.
- **Agente 3: interpretación:** redacta la respuesta final usando únicamente resultados ya calculados.

Esta separación permite diagnosticar fallos con precisión. Un caso puede detenerse por entrada inválida, configuración LLM, planificación incompleta, código no seguro, error de ejecución o fallo de interpretación.

5.3. Contrato del código generado

El contrato del código generado es la interfaz entre el modelo y la capa determinista. El script debe:

- definir `main()`;
- leer el payload mediante `Path(sys.argv[1]).read_text`;
- usar cargadores internos para los CSV;
- limitarse a importaciones permitidas;
- calcular métricas sobre datos históricos locales;
- devolver un único JSON por `stdout`;
- incluir limitaciones cuando una métrica no pueda calcularse.

El script no redacta la respuesta final. Su responsabilidad es producir resultados estructurados: métricas, tablas, datos visuales en JSON si se solicitan y avisos de limitación. El agente de interpretación se encarga después de convertir esa salida en lenguaje natural.

5.4. Gestión de errores

Los errores se separan por origen: entrada, configuración del modelo, formato JSON, seguridad del código, ejecución e interpretación. El sistema no oculta estos fallos bajo una respuesta aparente. Si no puede completar una fase, devuelve un estado controlado y conserva el detalle del error.

Esta gestión es especialmente importante en un sistema con generación de código. Un fallo de cuota o red no significa mala planificación financiera; un código rechazado no equivale a una mala interpretación; y una ejecución correcta puede requerir todavía una respuesta final más clara. Separar los estados permite evaluar el sistema como ingeniería, no sólo como conversación.

Capítulo 6

Implementación

6.1. Estructura del código

La implementación se organiza en módulos pequeños:

Módulo	Responsabilidad
<code>schemas.py</code>	Define la entrada, el plan interpretado, los artefactos y el estado del workflow.
<code>validation.py</code>	Valida query, tickers, fechas y rutas CSV antes de llamar al LLM.
<code>nodes.py</code>	Implementa los nodos del flujo: análisis LLM, codegen, seguridad, ejecución e interpretación.
<code>src/llm/</code>	Contiene prompts, cliente LLM compatible con Chat Completions y funciones del pipeline LLM.
<code>code_security.py</code>	Revisa el código generado antes de ejecutarlo.
<code>code_runner.py</code>	Guarda el script, lanza la ejecución y captura stdout, stderr y artefactos.
<code>market_data.py</code>	Proporciona funciones auxiliares para leer CSV financieros descargados con <code>yfinance</code> .
<code>tests/</code>	Comprueba el workflow con llamadas LLM simuladas y escenarios de error.

Tabla 6.1: Módulos principales de la implementación.

6.2. Repositorio y organización del trabajo

El desarrollo se ha gestionado en un repositorio GitHub disponible en <https://github.com/CarlosRuiz4102/tfm>. El uso de control de versiones permite conservar la evolución del código, la memoria, las evaluaciones y la documentación auxiliar en un único espacio de trabajo. Esta organización facilita revisar qué parte corresponde a datos, qué parte corresponde al flujo LLM, qué parte corresponde a resultados y qué parte corresponde a documentación académica.

La estructura del repositorio separa responsabilidades de forma cercana a una práctica MLOps ligera:

- **data/**: contiene el catálogo de casos, los CSV históricos congelados y sus metadatos.
- **src/**: agrupa el código de aplicación, dividido en esquemas, grafo, integración LLM y ejecución controlada.
- Automatizaciones reproducibles: permiten generar EDA, ejecutar evaluaciones y producir informes de apoyo.
- **tests/**: valida el workflow con respuestas LLM simuladas, evitando depender de APIs externas durante las pruebas básicas.
- Documentación y memoria: conserva la memoria LaTeX, figuras y registros de resultados.
- **results/**: almacena artefactos generados durante ejecuciones, como código generado, datos de entrada, registros y salidas JSON.

También se separan configuración y secretos. El fichero `.env` contiene claves locales y queda excluido del control de versiones, mientras que `.env.example` documenta las variables necesarias para reproducir la configuración sin exponer credenciales. Del mismo modo, `.gitignore` excluye entornos virtuales, cachés, registros y resultados generados de gran tamaño, manteniendo versionados los artefactos que aportan trazabilidad académica.

Aunque el proyecto no implementa un ciclo MLOps productivo completo con despliegue continuo, sí incorpora prácticas propias de ese enfoque: versionado del código, separación entre datos, código y resultados, evaluaciones repetibles, pruebas automatizadas, conservación de artefactos, documentación de configuraciones y registro explícito de fallos. Para el alcance del TFM, estas prácticas son suficientes para defender que el sistema no se limita a una demo puntual, sino que se ha organizado como un prototipo reproducible y auditable.

6.3. Contratos de datos

La entrada del sistema se representa mediante `FinancialQueryInput`. La planificación generada por el LLM se almacena en `AnalysisPlan`, que contiene objetivo interpretado, tipo de análisis, nivel A/B/C, métricas esperadas, requisitos de datos, requisitos de salida, preferencias de presentación y razonamiento resumido. El estado completo se conserva en `WorkflowState`.

La necesidad de funciones auxiliares de datos apareció durante las pruebas de desarrollo: los CSV generados por `yfinance` pueden contener cabeceras multinivel y formatos distintos según el activo o el intervalo. Para reducir errores repetidos en el código generado, se incorporaron funciones reutilizables de carga y normalización. Así, el LLM no tiene que resolver desde cero cada variante de lectura de CSV y puede concentrarse en el cálculo solicitado.

6.4. Pipeline LLM

El pipeline LLM se divide en tres llamadas principales. La primera interpreta la consulta y construye un plan estructurado. La segunda genera código Python usando tanto el plan como el mensaje original del usuario. La tercera interpreta la salida JSON producida por el script y redacta la respuesta final.

El prompt de generación de código exige que el script lea un payload por `argv[1]`, use librerías permitidas y escriba un único JSON en `stdout`. Esta restricción permite tratar el código generado como un artefacto ejecutable, auditable y validable.

El flujo completo puede resumirse como una cadena de transformación controlada:

prompt → código Python → validación de seguridad → ejecución →
JSON → interpretación final.

Esta cadena es importante porque el LLM no produce directamente una afirmación financiera final sin pasar antes por cálculos ejecutados. El primer bloque LLM prepara el análisis y genera el script; el entorno de ejecución produce datos estructurados; y el segundo bloque LLM redacta una explicación basada en esa salida.

Elemento	Contrato exigido
Entrada	El script recibe un único payload JSON por <code>argv[1]</code> , con consulta, tickers, fechas, intervalo y rutas CSV.
Lectura de datos	El código debe leer únicamente los CSV proporcionados por el workflow y no descargar datos externos.
Salida	Debe escribir un único objeto JSON en <code>stdout</code> , mediante <code>json.dumps</code> , para que el sistema pueda analizarlo.
Contenido	La salida debe incluir métricas, tablas o series estructuradas necesarias para responder a la consulta.
Artefactos	La ejecución conserva script, salida estándar, salida de error, código de retorno y metadatos de ejecución.
Errores	Si el script falla, el error queda registrado y el workflow puede devolver una explicación controlada o intentar reparación.

Tabla 6.2: Contrato operativo del código Python generado por el LLM.

6.5. Control de seguridad

Antes de ejecutar el script, el módulo `code_security.py` analiza el código con el AST de Python. El validador comprueba importaciones permitidas, llamadas bloqueadas como `eval` o `exec`, presencia de `main()` y uso de `json.dumps`. Si el código no cumple el contrato, el workflow termina con estado de error y no ejecuta el script.

El uso del árbol de sintaxis abstracta permite revisar la estructura del programa antes de ejecutarlo, en lugar de depender solo de búsquedas textuales. La validación comprueba nodos de importación, llamadas a funciones y elementos mínimos del contrato. Con ello se reduce el riesgo de que un script generado por LLM ejecute operaciones fuera del alcance del MVP. Esta capa no convierte el entorno en una sandbox completa, pero sí añade una barrera explícita entre generación y ejecución.

Comprobación	Finalidad
Imports permitidos	Limitar el código a librerías necesarias para análisis de datos y serialización.
Funciones bloqueadas	Evitar llamadas peligrosas o no necesarias, como <code>eval</code> y <code>exec</code> .
Función <code>main()</code>	Forzar una estructura ejecutable y sencilla de auditar.
Salida JSON	Garantizar que el resultado pueda pasar al nodo de interpretación sin ambigüedad.
Errores explícitos	Separar fallos de seguridad, fallos de ejecución y fallos de interpretación.

Tabla 6.3: Comprobaciones principales del validador de seguridad.

6.6. Reparación y compactación

Cuando el código generado no supera la validación o falla durante la ejecución, el workflow puede solicitar al LLM una reparación manteniendo la consulta original, el plan analítico y el detalle del error. La reparación no evita la validación: el código corregido vuelve a pasar por el mismo control de seguridad antes de ejecutarse. Si la reparación tampoco cumple el contrato, el caso se marca como error controlado.

En consultas de Nivel B o C, el script puede generar series largas. Estas series son útiles como artefacto completo, pero pueden saturar el contexto del agente de interpretación. Por ello, antes de la interpretación final se compactan listas y textos extensos: se conserva una muestra representativa para el LLM interpretador y la salida completa queda guardada como evidencia auditable. Esta decisión permite mantener trazabilidad sin degradar la respuesta final por exceso de contexto.

La decisión práctica fue limitar o resumir datos extensos enviados al agente de

interpretación, manteniendo siempre la salida completa en los registros. De este modo, las consultas de Nivel B y C pueden producir datos ricos sin convertir la interpretación final en una llamada demasiado grande para el modelo.

6.7. Pruebas

Las pruebas automatizadas simulan las respuestas LLM para evitar dependencias externas durante su ejecución. De este modo se validan el workflow, el control de seguridad, la ejecución controlada y los errores de entrada sin consumir APIs reales.

Capítulo 7

Integración de modelos de lenguaje

7.1. Papel del LLM en el sistema

El sistema incorpora modelos de lenguaje como componentes especializados dentro de un flujo de análisis financiero histórico. El modelo no se utiliza como fuente directa de precios, como asesor financiero ni como mecanismo de predicción. Su función es transformar lenguaje natural en planificación analítica, generar código bajo contrato y redactar una explicación final a partir de resultados ya calculados.

La arquitectura actual distingue tres agentes:

- **Agente 1: planificación.** Interpreta la petición del usuario, identifica el nivel de análisis A/B/C y define métricas, columnas, tablas o datos visuales que deben obtenerse.
- **Agente 2: generación de código.** Transforma el plan analítico en un script Python autocontenido, limitado por un contrato de entrada, salida e importaciones permitidas.
- **Agente 3: interpretación.** Recibe únicamente resultados calculados y redacta una explicación en español, separando datos históricos, lectura y limitaciones, sin recomendaciones de inversión.

Esta división evita que una única respuesta generativa mezcle planificación, cálculo y redacción. El modelo puede adaptar el sistema a consultas distintas, pero las cifras que llegan al usuario proceden de una ejecución programática sobre CSV locales.

7.2. Contrato de prompts

Los prompts se diseñan como contratos operativos. El agente planificador debe devolver JSON con el nivel A/B/C, el objetivo analítico, el tipo de análisis, las métricas

requeridas, las columnas necesarias y los requisitos de salida. No calcula cifras y no redacta conclusiones.

El agente generador devuelve un único campo `code`. El script generado debe leer un payload JSON por `argv[1]`, cargar los datos mediante utilidades autorizadas, calcular las métricas necesarias y escribir un único JSON por `stdout`. No puede descargar datos externos, crear procesos, usar rutas no recibidas ni imprimir texto libre. Si la consulta pide tablas o datos visuales, el script debe devolverlos como estructuras JSON, no como imágenes.

El agente interpretador recibe la salida ejecutada y el plan. Su prompt prohíbe recalcular, completar cifras ausentes o añadir información externa. La respuesta debe ajustar su profundidad al nivel A/B/C, explicar las métricas observadas y declarar limitaciones cuando existan datos insuficientes.

Agente	Entrada principal	Salida exigida
Planificación	Consulta validada, tickers, fechas, CSV y reglas A/B/C.	Plan JSON con nivel, objetivo, métricas, columnas, requisitos de datos y requisitos de salida.
Generación de código	Consulta original, entrada normalizada y plan analítico.	JSON con <code>code</code> ; el script calcula sobre CSV locales y devuelve resultados estructurados.
Interpretación	Salida ejecutada y plan analítico.	Respuesta final en español, fiel a los resultados, prudente y sin recomendación de inversión.

Tabla 7.1: Responsabilidades de los tres agentes LLM.

7.3. Niveles A/B/C

El nivel de análisis no se recibe como etiqueta externa. Lo decide el agente de planificación a partir de la consulta:

- **Nivel A:** petición directa, sin formato especial. Requiere métricas esenciales y una respuesta breve.
- **Nivel B:** análisis más completo, comparación clara, tabla o una salida visual principal. Requiere varias métricas y estructura tabular cuando proceda.
- **Nivel C:** petición profesional, multicriterio o con formato impuesto. Requiere varias familias de métricas, tablas, datos visuales si se solicitan y limitaciones explícitas.

Esta graduación permite exigir más al modelo sin convertir todas las consultas en informes extensos. También facilita la evaluación: un fallo puede localizarse en la planificación del nivel, en el código generado, en la ejecución o en la interpretación.

7.4. Cliente compatible con Chat Completions

La integración usa un cliente compatible con la interfaz de Chat Completions. OpenAI denomina *Chat Completions* al endpoint que genera una respuesta del modelo a partir de una lista de mensajes que forman una conversación [19]. En este proyecto, la expresión describe el formato de interacción basado en mensajes y la forma de invocar el modelo desde el cliente.

El modelo principal del sistema es `openai/gpt-oss-20b` servido sobre vLLM. vLLM actúa como servidor de inferencia compatible con la interfaz de mensajes, mientras que el modelo evaluado y documentado sigue siendo `openai/gpt-oss-20b`. Esta distinción evita confundir el modelo con la infraestructura que lo expone.

7.5. Gestión de errores

El flujo registra errores de configuración, red, cuota, formato JSON, validación de código, ejecución e interpretación. Cada tipo de error se conserva en el estado del workflow para distinguir problemas de entorno, problemas del modelo y problemas del contrato técnico.

Si el agente de planificación o generación devuelve JSON no parseable, el sistema solicita una nueva respuesta estructurada. Si el código falla o no supera la validación de seguridad, puede pedirse una reparación al modelo, pero el script reparado vuelve a pasar por los mismos controles antes de ejecutarse. Esta decisión evita que la reparación sea una vía para saltarse el contrato.

7.6. Credenciales y despliegue

Las credenciales se gestionan mediante variables de entorno locales. El fichero de configuración real no debe versionarse ni compartirse. La configuración mínima requiere la URL del servidor vLLM, una clave del endpoint y el nombre `openai/gpt-oss-20b`. El código acepta también alias compatibles con OpenAI para reutilizar el SDK, pero la configuración de referencia del proyecto es vLLM.

Capítulo 8

Resultados y evaluación

8.1. Objetivo

La evaluación comprueba si el prototipo mantiene el flujo completo al aumentar la dificultad de las consultas. No busca demostrar capacidad predictiva ni establecer un ranking de servicios LLM. El foco está en verificar planificación, generación de código, validación, ejecución e interpretación final sobre datos históricos locales.

8.2. Batería A/B/C

La batería principal contiene 15 consultas equilibradas: cinco de Nivel A, cinco de Nivel B y cinco de Nivel C. La dificultad asciende dentro de cada nivel.

Nivel	Progresión de dificultad	Comprobación principal
A	Crecimiento simple, comparación básica, visión descriptiva, retornos y riesgo básico.	Que el sistema responda consultas directas sin sobrecargar la salida.
B	Análisis ampliado, comparación normalizada, retorno-riesgo, activo volátil e intradía.	Que el sistema produzca métricas y estructura tabular o visual cuando se pide más detalle.
C	Informe comparativo, ranking multicriterio, índice amplio, materia prima intradía y formato de estrés.	Que el sistema coordine varias métricas y restricciones simultáneas.

Tabla 8.1: Estructura de la batería A/B/C de 15 consultas.

La quinta consulta de Nivel C fuerza cuatro bloques de salida: resumen de métricas, clasificación mensual, datos normalizados y datos de drawdown. Su objetivo es presionar la planificación y el código generado, no producir una figura en la memoria.

8.3. Métricas técnicas

La evaluación se apoya en métricas técnicas registrables:

- **Compleitud:** estado final `completed`.
- **Plan generado:** existencia de un plan A/B/C parseable.
- **Código generado:** existencia de script que cumple el contrato mínimo.
- **Código validado:** superación de la revisión de seguridad.
- **Ejecución correcta:** retorno cero y salida JSON parseable.
- **Interpretación final:** respuesta en español basada en resultados calculados.
- **Prudencia financiera:** ausencia de recomendaciones de compra o venta.
- **Tiempo de ejecución:** duración observada por caso.

Las métricas subjetivas, como claridad percibida, utilidad o calidad comunicativa, quedan previstas para una fase posterior. Se consideran relevantes, pero deben reforzarse con una rúbrica más estable y revisores consistentes antes de tratarlas como evidencia principal.

8.4. Resultados observados

La versión actual del prototipo se orienta a `openai/gpt-oss-20b` servido sobre vLLM. vLLM se interpreta como el servidor de despliegue compatible, no como un proveedor alternativo del estudio.

La evaluación registra para cada caso si el flujo llega a planificar, generar código, validar, ejecutar e interpretar. Los casos parciales son informativos porque permiten localizar el punto de fallo. Por ejemplo, un caso con plan correcto pero ejecución fallida señala un problema distinto a un caso que no llega a producir JSON válido en la primera llamada LLM.

Resultado registrado	Interpretación
<code>completed</code>	El flujo alcanza la respuesta final con salida ejecutada.
<code>completed_with_error</code>	El sistema devuelve un error controlado y no inventa una respuesta alternativa.
<code>code_rejected</code>	El script no supera el contrato o la validación de seguridad.
<code>execution_failed</code>	El script era ejecutable pero falló al calcular o serializar la salida.
Avisos de reparación	El sistema necesitó una segunda llamada para corregir código o formato.

Tabla 8.2: Lectura de los estados principales de evaluación.

8.5. Limitaciones

Una única pasada de evaluación no permite afirmar robustez estadística. La disponibilidad del modelo, la cuota, la latencia y la estabilidad del endpoint pueden afectar a los resultados. Además, la generación de código mediante LLM requiere validación estricta porque una solución plausible puede fallar por detalles de formato, columnas, datos insuficientes o serialización.

El sistema trabaja con datos históricos locales. No incorpora noticias, datos fundamentales ni precios actuales externos. Por tanto, las respuestas deben leerse como análisis descriptivos del periodo disponible, no como predicciones ni recomendaciones de inversión.

8.6. Síntesis

La evaluación muestra un prototipo funcional y trazable. La contribución principal es que el sistema no mide sólo si el texto final parece útil, sino si cada fase produce artefactos verificables: plan, código, validación, ejecución y explicación. La batería A/B/C permite aumentar la exigencia de forma ordenada y deja preparadas métricas subjetivas que podrán reforzarse en trabajo futuro.

Capítulo 9

Aspectos éticos, seguridad y uso responsable

9.1. Delimitación financiera del sistema

El sistema se orienta exclusivamente al análisis financiero histórico y descriptivo. Sus respuestas deben interpretarse como explicaciones basadas en datos pasados, no como predicciones ni como recomendaciones de inversión. Esta delimitación forma parte del diseño del MVP y debe mantenerse visible tanto en la memoria como en cualquier interfaz futura.

El prototipo calcula métricas como crecimiento, retornos, volatilidad, drawdown e indicadores técnicos básicos. No realiza predicciones, no recomienda comprar o vender activos y no adapta decisiones a perfiles personales de riesgo.

9.2. Riesgos de interpretación

Un riesgo relevante es que el usuario confunda una explicación descriptiva con asesoramiento financiero. Para reducirlo, el sistema debe utilizar formulaciones prudentes, explicitar el período analizado y recordar que los resultados históricos no garantizan comportamientos futuros.

La capa LLM aumenta este riesgo si redacta respuestas demasiado persuasivas. Por ello, la configuración actual pide al modelo interpretar resultados ya calculados, no añadir datos externos y evitar recomendaciones de inversión.

9.3. Seguridad en la generación y ejecución de código

La generación automática de código introduce riesgos específicos. Un script mal generado podría fallar, calcular una métrica incorrecta o acceder a recursos no previs-

tos. En este proyecto, el riesgo se reduce mediante entradas estructuradas, ejecución controlada, validación mínima del contrato y captura explícita de salidas.

El código generado se ejecuta como proceso externo, con registro de salida estándar, salida de error y código de retorno. Esta estrategia permite revisar errores y evita que la respuesta final oculte fallos de ejecución.

9.4. Privacidad y datos

Los datos utilizados son series históricas de mercado y no contienen datos personales. Aun así, las claves de API del modelo se consideran información sensible y se almacenan exclusivamente en el fichero local `.env`, excluido del control de versiones.

En una versión productiva, sería necesario revisar las condiciones de uso del proveedor de datos y del modelo, así como las políticas de retención de información enviada a APIs externas.

9.5. Sostenibilidad, costes y recursos

El enfoque incremental reduce costes computacionales. El MVP no entrena modelos desde cero y puede ejecutarse en CPU sobre datos ya descargados. La capa LLM se concentra en planificación, generación de código e interpretación, sin entrenar ni ajustar pesos.

Esta estrategia también mejora la sostenibilidad del desarrollo: evita ejecuciones innecesarias de modelos grandes, reduce dependencia de infraestructura GPU propia y permite mantener una configuración de uso continuado más controlada.

9.6. Limitaciones de uso

El sistema no debe utilizarse como herramienta profesional de decisión financiera sin validaciones adicionales. Su valor actual es académico y experimental: demostrar una arquitectura modular, trazable y robusta para automatizar análisis históricos.

Entre las limitaciones principales se encuentran el uso de un conjunto de datos acotado, la dependencia de datos obtenidos desde Yahoo Finance mediante `yfinance` [17], [18], la ausencia de validación financiera profesional y la no incorporación de información fundamental, noticias o variables macroeconómicas.

Capítulo 10

Conclusiones y trabajo futuro

El trabajo desarrollado demuestra la viabilidad de un sistema LLM para análisis financiero histórico basado en una arquitectura modular, trazable y extensible. El prototipo valida entradas estructuradas, planifica el análisis, genera código Python, revisa su seguridad, ejecuta el script sobre CSV locales y redacta una respuesta final a partir de resultados observados.

La aportación principal no es predecir el mercado, sino conectar lenguaje natural, cálculo reproducible y explicación financiera prudente. Para ello se separan tres agentes: planificación, generación de código e interpretación. Esta división permite revisar el sistema por partes y detectar si un fallo procede de la comprensión de la consulta, del código, de la ejecución o de la redacción final.

Los objetivos planteados se cumplen en una versión funcional y evaluable. Se ha definido un workflow modular, se ha incorporado generación de código mediante LLM, se ha añadido una capa de seguridad previa a la ejecución, se han conservado artefactos auditables y se ha preparado una batería A/B/C con cinco consultas por nivel.

La configuración actual se centra en `openai/gpt-oss-20b` servido sobre vLLM. vLLM se entiende como infraestructura de despliegue para servir el modelo mediante un endpoint compatible. Esta distinción evita confundir el modelo evaluado con el servidor que lo expone.

El prototipo trabaja con un conjunto acotado de datos históricos ya descargados. No aborda predicción futura, recomendación de inversión, análisis fundamental ni incorporación de noticias. La generación de código mediante LLM queda limitada por contratos, validaciones y revisión posterior, por lo que requiere más pruebas antes de cualquier uso fuera del entorno académico.

Como trabajo futuro se proponen varias líneas: ampliar la batería de estrés, reforzar la validación del código generado, ejecutar varias rondas para reducir variabilidad, incorporar más activos y ventanas temporales, y diseñar métricas subjetivas más sólidas para valorar claridad, utilidad y adecuación de la respuesta al nivel A/B/C. Estas métricas subjetivas deberán apoyarse en una rúbrica más explícita antes de usarse

como evidencia principal.

En conjunto, el TFM se encuentra en un punto sólido para memoria: existe un MVP ejecutable, una metodología defendible y una arquitectura que transforma consultas financieras históricas en resultados calculados y explicaciones trazables, manteniendo siempre la separación entre rendimiento histórico observado e interpretación.

Referencias

- [1] A. Vaswani et al., «Attention Is All You Need,» en *Advances in Neural Information Processing Systems*, 2017. dirección: <https://arxiv.org/abs/1706.03762>
- [2] Analytics Vidhya, *60+ Generative AI Terms You Must Know*, 2024. visitado 25 de mayo de 2026. dirección: <https://www.analyticsvidhya.com/blog/2024/01/generative-ai-terms/>
- [3] D. Araci, «FinBERT: Financial Sentiment Analysis with Pre-trained Language Models,» *arXiv preprint arXiv:1908.10063*, 2019. DOI: 10.48550/arXiv.1908.10063 arXiv: 1908.10063 [cs.CL]. dirección: <https://arxiv.org/abs/1908.10063>
- [4] S. Wu et al., «BloombergGPT: A Large Language Model for Finance,» *arXiv preprint arXiv:2303.17564*, 2023. DOI: 10.48550/arXiv.2303.17564 arXiv: 2303.17564 [cs.LG]. dirección: <https://arxiv.org/abs/2303.17564>
- [5] H. Yang, X.-Y. Liu y C. D. Wang, «FinGPT: Open-Source Financial Large Language Models,» *arXiv preprint arXiv:2306.06031*, 2023. DOI: 10.48550/arXiv.2306.06031 arXiv: 2306.06031 [q-fin.CP]. dirección: <https://arxiv.org/abs/2306.06031>
- [6] T. Zhou, P. Wang, Y. Wu y H. Yang, «FinRobot: AI Agent for Equity Research and Valuation with Large Language Models,» *arXiv preprint arXiv:2411.08804*, 2024. DOI: 10.48550/arXiv.2411.08804 arXiv: 2411.08804 [q-fin.CP]. dirección: <https://arxiv.org/abs/2411.08804>
- [7] Y. Xiao, E. Sun, D. Luo y W. Wang, «TradingAgents: Multi-Agents LLM Financial Trading Framework,» *arXiv preprint arXiv:2412.20138*, 2024. DOI: 10.48550/arXiv.2412.20138 arXiv: 2412.20138 [q-fin.TR]. dirección: <https://arxiv.org/abs/2412.20138>
- [8] S. Yao et al., «ReAct: Synergizing Reasoning and Acting in Language Models,» en *International Conference on Learning Representations*, 2023. arXiv: 2210.03629 [cs.CL]. dirección: <https://arxiv.org/abs/2210.03629>

- [9] T. Schick et al., «Toolformer: Language Models Can Teach Themselves to Use Tools,» *arXiv preprint arXiv:2302.04761*, 2023. DOI: 10.48550/arXiv.2302.04761 arXiv: 2302.04761 [cs.CL]. dirección: <https://arxiv.org/abs/2302.04761>
- [10] L. Wang et al., «A Survey on Large Language Model based Autonomous Agents,» *arXiv preprint arXiv:2308.11432*, 2023. DOI: 10.48550/arXiv.2308.11432 arXiv: 2308.11432 [cs.AI]. dirección: <https://arxiv.org/abs/2308.11432>
- [11] M. Chen et al., «Evaluating Large Language Models Trained on Code,» *arXiv preprint arXiv:2107.03374*, 2021. DOI: 10.48550/arXiv.2107.03374 arXiv: 2107.03374 [cs.LG]. dirección: <https://arxiv.org/abs/2107.03374>
- [12] Y. Lai et al., «DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation,» *arXiv preprint arXiv:2211.11501*, 2022. DOI: 10.48550/arXiv.2211.11501 arXiv: 2211.11501 [cs.SE]. dirección: <https://arxiv.org/abs/2211.11501>
- [13] J.-B. Alayrac et al., «Flamingo: A Visual Language Model for Few-Shot Learning,» en *Advances in Neural Information Processing Systems*, 2022. dirección: <https://arxiv.org/abs/2204.14198>
- [14] OpenAI, *GPT-4V(ision) System Card*, 2023. visitado 3 de jun. de 2026. dirección: <https://openai.com/index/gpt-4v-system-card/>
- [15] Z. Ji et al., «Survey of Hallucination in Natural Language Generation,» *ACM Computing Surveys*, 2022. DOI: 10.1145/3571730 arXiv: 2202.03629 [cs.CL]. dirección: <https://arxiv.org/abs/2202.03629>
- [16] P. Lewis et al., «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,» en *Advances in Neural Information Processing Systems*, 2020. arXiv: 2005.11401 [cs.CL]. dirección: <https://arxiv.org/abs/2005.11401>
- [17] R. Aroussi, *yfinance Python Library*, 2026. visitado 25 de mayo de 2026. dirección: <https://github.com/ranaroussi/yfinance>
- [18] Yahoo Finance Help, *Download historical data in Yahoo Finance*, 2026. visitado 25 de mayo de 2026. dirección: <https://help.yahoo.com/kb/finance/historical-prices-adjusted-close-sln2311.html>
- [19] OpenAI, *Chat Completions API Reference*, 2026. visitado 4 de jun. de 2026. dirección: <https://platform.openai.com/docs/api-reference/chat/create-chat-completion>
- [20] LangChain, «LangGraph: Building Stateful, Multi-Actor Applications with LLMs,» 2024. dirección: <https://langchain-ai.github.io/langgraph/>

- [21] AI@Meta, «The Llama 3 Herd of Models,» *arXiv preprint arXiv:2407.21783*, 2024. DOI: 10.48550/arXiv.2407.21783 arXiv: 2407.21783 [cs.AI]. dirección: <https://arxiv.org/abs/2407.21783>
- [22] OpenAI, *gpt-oss-120b & gpt-oss-20b Model Card*, 2025. DOI: 10.48550/arXiv.2508.10925 arXiv: 2508.10925 [cs.CL]. dirección: <https://arxiv.org/abs/2508.10925>
- [23] OpenAI, *OpenAI o1 System Card*, 2024. DOI: 10.48550/arXiv.2412.16720 arXiv: 2412.16720 [cs.AI]. dirección: <https://arxiv.org/abs/2412.16720>
- [24] DeepSeek-AI, «DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,» *arXiv preprint arXiv:2501.12948*, 2025. DOI: 10.48550/arXiv.2501.12948 arXiv: 2501.12948 [cs.CL]. dirección: <https://arxiv.org/abs/2501.12948>
- [25] U.S. Securities and Exchange Commission, *Investor Bulletin: Performance Claims*, 2022. visitado 27 de mayo de 2026. dirección: <https://www.investor.gov/introduction-investing/general-resources/news-alerts/alerts-bulletins/investor-bulletins-47>
- [26] Investor.gov, *What is Risk?* 2026. visitado 27 de mayo de 2026. dirección: <https://www.investor.gov/introduction-investing/investing-basics/what-risk>
- [27] CFA Institute, *Portfolio Risk and Return: Part I*, 2026. visitado 27 de mayo de 2026. dirección: <https://www.cfainstitute.org/insights/professional-learning/refresher-readings/2026/portfolio-risk-return-part-1>
- [28] CFA Institute, *Performance Drawdowns in Asset Management: Extending Draw-down Analysis to Active Returns*, 2018. visitado 27 de mayo de 2026. dirección: <https://rpc.cfainstitute.org/research/cfa-digest/2018/01/dig-v48-n1-3>
- [29] Atlassian, *A Complete Guide to Line Charts*, 2026. visitado 27 de mayo de 2026. dirección: <https://www.atlassian.com/data/charts/line-chart-complete-guide>

Apéndice A

Anexos

A.1. Ejecución del MVP

El prototipo puede ejecutarse con ejemplos incluidos en el repositorio:

```
python run_mvp.py --example growth_nvda
python run_mvp.py --example compare_nvda_amd
python run_mvp.py --example overview_aapl
python run_mvp.py --example returns_qqq_spy
python run_mvp.py --example risk_qqq_spy
python run_mvp.py --example technical_aapl
```

También puede recibir un JSON de entrada con consulta, tickers, fechas o periodo, intervalo y rutas a CSV locales.

A.2. Configuración de API

Las claves de API se guardan localmente en el fichero `.env`, que no debe subirse al repositorio. La configuración mínima es:

```
OPENAI_API_KEY=...
VLLM_BASE_URL=https://servidor-vllm.example/v1
VLLM_API_KEY=...
VLLM_MODEL=openai/gpt-oss-20b
```

Si el modelo de OpenAI se sirve desde un servidor vLLM compatible con la interfaz de mensajes, puede indicarse una URL base:

```
OPENAI_BASE_URL=https://servidor-vllm.example/v1
OPENAI_MODEL=openai/gpt-oss-20b
```

En este caso, vLLM describe la infraestructura que sirve el modelo; el modelo sigue identificándose como modelo de OpenAI [22].

A.3. Batería de consultas A/B/C

La batería contiene 15 consultas equilibradas: cinco de Nivel A, cinco de Nivel B y cinco de Nivel C. Dentro de cada nivel, los ejemplos aumentan progresivamente su dificultad.

Tabla A.1: Batería de 15 consultas por nivel de profundidad.

Nivel	Consulta resumida	Objetivo de evaluación
A	Crecimiento de Nvidia en los últimos cinco años.	Crecimiento histórico básico.
A	Comparación simple de Nvidia y AMD en dos años.	Comparación por rentabilidad.
A	Visión general de AAPL en tres meses.	Resumen descriptivo.
A	Retornos de QQQ y SPY en 2024.	Retornos históricos resumidos.
A	Riesgo histórico de QQQ y SPY en 2024.	Riesgo, volatilidad y extremos.
B	Análisis ampliado de Nvidia con tabla y datos de evolución.	Salida estructurada de un activo.
B	Comparación de QQQ y SPY con tabla y serie normalizada.	Comparación relativa base 100.
B	Retorno y riesgo de AAPL con explicación sencilla.	Relación retorno-riesgo.
B	Bitcoin 2024 con rentabilidad, volatilidad y drawdown.	Cobertura de activo volátil.
B	EUR/USD intradía con variación, rango y volatilidad horaria.	Cobertura de divisa horaria.
C	Informe QQQ/SPY con tabla, normalización y drawdown.	Informe comparativo completo.
C	Ranking NVDA/AMD por rentabilidad, riesgo y drawdown.	Comparación multicriterio.
C	Informe detallado del S&P 500 desde 2020.	Serie larga de índice amplio.
C	Informe horario del oro con mejores/peores intervalos y rango.	Materia prima e intervalo horario.
C	QQQ/SPY 2024 con cuatro bloques obligatorios.	Estrés de formato y planificación.

A.4. Prompts usados por los agentes

El prompt base define un componente profesional de análisis financiero histórico. Todos los agentes comparten las mismas restricciones: trabajar sólo con datos históricos disponibles en CSV, no predecir, no recomendar inversión y no añadir información externa.

A.4.1. Agente 1: planificación

El agente planificador devuelve JSON con nivel A/B/C, objetivo analítico, tipo de análisis, métricas, columnas requeridas, requisitos de datos, requisitos de salida y una justificación breve del nivel elegido. No calcula cifras y no redacta la respuesta final.

A.4.2. Agente 2: generación de código

El agente generador devuelve un JSON con un único campo `code`. El script debe leer el payload desde `argv[1]`, cargar datos mediante funciones autorizadas, calcular métricas sobre CSV locales y emitir un único JSON por `stdout`. Si se solicitan datos visuales, se devuelven como estructuras JSON, no como ficheros de imagen.

A.4.3. Agente 3: interpretación

El agente interpretador recibe la salida ejecutada y el plan. Su tarea es explicar los resultados en español, ajustando la profundidad al nivel A/B/C y separando datos históricos, lectura y limitaciones. No recalcula ni incorpora datos externos.

A.5. Trazabilidad de una ejecución

Cada ejecución conserva la entrada normalizada, el plan analítico, el código generado, la salida estándar, los errores y la respuesta final. Estos artefactos permiten repetir una prueba, revisar qué entendió cada agente y comprobar si la interpretación final se corresponde con los resultados calculados.

A.6. Métricas subjetivas futuras

La revisión subjetiva de calidad se mantiene como línea futura. La plantilla prevista incluye comprensión de la consulta, selección de métricas, adecuación del código, fidelidad a resultados, claridad, utilidad, reconocimiento de limitaciones, ausencia de recomendación de inversión y trazabilidad. Antes de usar estas métricas como evidencia principal será necesario reforzar la rúbrica y definir criterios de revisión más estables.